

moștenire

Șapte exerciții — adaugi tipuri noi peste clasele tale din `vanilla-ts` · TypeScript · Andreea

Unde lucrezi: `vanilla-ts/src/exercitii/`, în fișierele pe care le-ai scris la exercițiile 01–06. Fiecare exercițiu spune fișierul. Codul de rulare îl adaugi sub cel existent; ce afișa fișierul înainte trebuie să afișeze identic și după.

Nou, ai voie: `extends`, `super(...)` în constructor, `super.metoda()`, `protected`.

Încă NU folosim: `abstract` nou (ce e deja abstract din 06 rămâne), `interface`, `instanceof`, `static`, `any`, cast cu `as`.

Cum se lucrează: fiecare exercițiu îți cere să **construiești** ceva — o clasă nouă, o regulă pusă peste alta, o clasă mutată în mijlocul ierarhiei. Rândul **Gata când** spune cum verifici; se verifică prin **rulare** (`npx tsx`) și prin `npx tsc --noEmit` fără erori, nu prin citirea codului.

Partea 1 — adaugi un tip nou într-o ierarhie

1. Raportul cu dată

nivel 1

Fișierul `03-raport`. Adaugă **RaportCuData**, care moștenește `Raport` și primește la construire titlul și data (text, de forma `2026-10-02`). Textul lui începe cu linia `Generat: 2026-10-02`, urmată de tot ce afișa raportul de bază.

Nu rescrii compunerea titlului și a liniilor; folosești ce face deja baza.

Rulare: titlul `Rezultate suita`, data `2026-10-02`, liniile `3 teste verzi` și `1 test rosu`.

Gata când: ies patru linii, prima fiind `Generat: 2026-10-02`; iar ieșirea lui `RaportSemnat` de deasupra e neschimbată.

De reținut: subclasa adaugă, nu rescrie. `super.text()` e locul în care refolosești, nu copiezi.

2. Numărătorul cu pas

nivel 1

Fișierul `04-numarator`. Adaugă **NumaratorCuPas**, care moștenește `Numarator` și primește la construire un pas; `urmatorul()` crește valoarea cu pasul, nu cu 1. `NumaratorCuLimita` rămâne exact cum e.

Rulare: pasul 5, trei apeluri, rezultatele pe o linie.

Gata când: apare `5 10 15`, iar liniile vechi `1 2 3` și `1 2 2 2` sunt identice cu cele de dinainte.

De reținut: două subclase ale aceleiași baze nu știu una de alta. Ai adăugat un tip fără să atingi nici baza, nici sora lui.

Partea 2 — subclasa pune o regulă peste bază

3. Cheia nu se repetă

mai greu

Fișierul `01-set-de-date`. Adaugă **SetDeDateCuCheie**, care moștenește `SetDeDate` și primește la construire coloanele și numele coloanei-cheie. `adauga` refuză un rând a cărui valoare din coloana-cheie există deja, cu `Error("Duplicate key ESID-100")`. Rândurile rămân ținute în bază; regula bazei (numărul de valori) rămâne valabilă.

Ai de luat o decizie: subclasa trebuie să vadă ce rânduri există deja. Fie trece prin metoda publică `valorile(...)`, fie rândurile devin `protected`. Alege și scrie într-un rând de ce.

Rulare: cheia `ESID`, cele trei rânduri de la 01, apoi un al patrulea `ESID-100 / FR-X / Blocked` într-un `try / catch`.

Gata când: apare `Duplicate key ESID-100`, `numarRanduri()` rămâne 3, iar un rând cu două valori dă în continuare `Row has 2 values, expected 3`.

De reținut: un override care nu cheamă `super.adauga(...)` pierde regulile bazei. Le pierzi în tăcere: nimic nu te anunță.

4. Nicio valoare goală

mai greu

Fișierul `01-set-de-date`. Adaugă **SetDeDateStrict**, care moștenește `SetDeDate` și refuză rândul dacă vreo valoare e goală după tăierea spațiilor, cu `Error("Empty value in column STATUS")` (numele coloanei reale).

Proba de ordine: dă-i un rând cu **două** valori, dintre care una goală. Ce mesaj iese: al tău sau al bazei? Hotărăște care regulă se verifică prima și scrie într-un rând de ce.

Gata când: fiecare mesaj apare la cazul lui, mesajul de la proba de ordine e cel pe care l-ai hotărât tu, iar `numarRanduri()` nu crește la niciun refuz.

De reținut: `super.adauga(rand)` e un apel ca oricare altul. Îl pui înainte sau după regula ta, și ordinea se vede în rezultat.

Partea 3 — provocări

5. Numărătorul ciclic, al treilea etaj

provocare

Fișierul `04-numarator`. Adaugă **NumaratorCiclic**, care moștenește `NumaratorCuLimita`: după ce a ajuns la limită, următorul apel revine la 1 și o ia de la capăt.

Constrângere: nu modifici nimic în `Numarator` și `NumaratorCuLimita`, și nu rescrii logica de limită. O folosești.

Rulare: limita 3, șapte apeluri.

Gata când: apare `1 2 3 1 2 3 1`, iar cele două clase de deasupra sunt neatinse.

De reținut: `super.urmatorel()` din subclasă cheamă etajul de deasupra, care la rândul lui cheamă etajul lui. Fiecare etaj adaugă o regulă și o lasă pe a celorlalți în picioare.

6. Raportul compus, fără moștenire

provocare

Fișierul `03-raport`. Scrie **RaportCompus**, care ține **două** rapoarte de orice fel (`Raport` , `RaportSemnat` , `RaportCuData`) și are `text()` = textul primului, o linie `---` , textul celui de-al doilea. Clasa **nu** moștenește `Raport` .

Scrie în două rânduri: de ce nu `extends Raport` aici? Ce ar fi moștenit degeaba?

Gata când: afișezi un `RaportSemnat` și un `RaportCuData` unul sub altul, cu `---` între ele, iar în clasa nouă nu apare cuvântul `extends` .

De reținut: moștenirea spune „este un”, compoziția spune „are un”. Când iei moștenire doar ca să refolosești o metodă, plătești cu câmpuri și metode pe care nu le vrei.

7. Clasa din mijloc, pentru exportatoarele cu separator

provocare

Fișierul `06-exportatoare`. `ExportatorCsv` și `ExportatorTabelText` fac același lucru cu alt separator; `ExportatorJson` nu.

Bagă între `Exportator` și cele două o clasă intermediară care ține separatorul (`protected` , dat prin constructor) și scrie `converteste` o singură dată. Cele două rămân doar cu extensia și cu apelul de constructor. `ExportatorJson` rămâne direct sub `Exportator` ; scrie într-un rând de ce nu încape în clasa din mijloc.

Proba: adaugă **ExportatorTsv** (separator tab, extensie `tsv`).

Gata când: `converteste` apare în exact două locuri (clasa din mijloc și JSON); ieșirea pentru csv/json/txt e identică cu cea de la 06; iar `ExportatorTsv` a încăput în cel mult patru linii.

De reținut: clasa din mijloc ține ce au în comun *doar unele* subclase. Nu urci în bază ce nu e al tuturor.